

**Московский государственный технический
университет им. Н.Э. Баумана**

**Факультет «Радиотехнический»
Кафедра «Системы обработки информации и управления»**

Курс «Парадигмы и конструкции языков программирования»

Отчет по домашнему заданию

Выполнил:
студент группы РТ5-31Б:
Кузнецов С. А.

Подпись и дата:

Проверил:
преподаватель кафедры ИУ5
Гапанюк Ю. Е.

Подпись и дата:

Москва, 2024 г.

Постановка задачи

Разработать бота для Telegram, способного генерировать текстовые сообщения на базе корпуса и цепей Маркова.

Текст программы

Файл «markovfrominput.py»:

```
import markovify
import re

# В этом файле лежит вся логика, касающаяся работы генератора текста, к которому
# обращается бот.
# model - обработанный корпус текста, используемый ботом.

class PostText(markovify.Text):
    """
    Подкласс от класса markovify.Text. Создаёт корпус, полагая концом "предложе-
    ния"
    в корпусе условный символ "¶" ("конец поста").
    """

    # Сейчас будем расшифровывать кое-что из родительского класса
    # reject_pat = re.compile(r"^(^)|('$)|\s'|'\s|[\\"(\(\)\[\]])")
    # "(начало строки ' ) ИЛИ ( ' конец строки) ИЛИ пробельный символ ' ИЛИ ' про-
    # бельный символ ИЛИ
    # ОДИН ИЗ СПИСКА: [", ( (, ), [ , ])]

    def sentence_split(self, text):
        return re.split(r"\s*¶\s*", text)

    # Регулярное выражение значит следующее: "Ноль и более пробельных символов,
    "конец поста", ноль и более пробельных символов"
    # (по-моему) (вроде как)

    def make_post(self, init_state=None, **kwargs):
        temp_sentence = self.make_sentence(init_state, **kwargs)
        if temp_sentence:
            while temp_sentence.count("#Роботник") > 1:
                temp_sentence = temp_sentence.replace(
                    "#Роботник", "", 1
                ) # Удаляем дубликаты хештега
            temp_sentence = temp_sentence.replace("#Роботник", "\n#Роботник")
            while temp_sentence.count("#ИИ5") > 1:
                temp_sentence = temp_sentence.replace("#ИИ5", "", 1)
            temp_sentence = temp_sentence.replace("#ИИ5", "\n#ИИ5")
            temp_sentence = temp_sentence.replace("П:", "\nП:")
            temp_sentence = temp_sentence.replace("С:", "\nС:")
```

```

        return temp_sentence
    else:
        return None

with open("sourcetext.txt", "r") as f:
    text = f.read()

model = PostText(text)

```

Файл «main.py»:

```

import telebot
import markovfrominput
import random

bot_token = "[ДАННЫЕ УДАЛЕНЫ]" # На месте "[ДАННЫЕ УДАЛЕНЫ]" должен быть действительный
#token для бота Telegram

bot = telebot.TeleBot(bot_token, parse_mode=None)

# Команда /about
@bot.message_handler(commands=["start", "about"])
def explain_yourself(message):
    bot.send_message(
        message.chat.id,
        "Привет, человек. Я - передатчик, получающий отрывки разговоров из аудиторий кафедры ИИ5.\n\n...такой кафедры не существует, говоришь? "
        "Ты точно живёшь в 2124 году? "
        "\n\nВ любом случае, если ты хочешь воспользоваться моими услугами, напиши /generate!",
    )

# Команда /help
@bot.message_handler(commands=["help"])
def explain_yourself(message):
    bot.send_message(
        message.chat.id,
        "Вот команды, которые я понимаю:\n"
        "/help - вывести этот текст.\n"
        '/generate - генерация цитаты единственного и неповторимого к.т.н. Роботника (или его неуловимых коллег типа "Роботника-младшего").\n'
        "/about - узнать, кто я такой.",
    )

# Команда /generate
@bot.message_handler(commands=["generate"])
def generate_command(message):

```

```

generated_post = markovfrominput.model.make_post(
    min_words=15, well_formed=False, state_size=2
)
if generated_post:
    bot.reply_to(message, generated_post)
else:
    bot.send_message(
        message.chat.id,
        "Сильная интерференция сигнала... Простите, попробуйте снова.",
    )

# Реакция на фото
@bot.message_handler(content_types=["photo"])
def handle_docs_audio(message):
    bot.send_message(
        message.chat.id,
        "Не понимаю, блокирую!",
    )

# Реакция на стикеры
@bot.message_handler(content_types=["sticker"])
def handle_docs_audio(message):
    bot.send_message(
        message.chat.id,
        "Привет, я тебя узнал!\n\nIP. 92.28.211.234\n"
        "N: 43.7462\n"
        "W: 12.4893\n"
        "SS Number: 6979191519182016\n"
        "IPv6: fe80::5dcd::ef69::fb22::d9888%12\n"
        "Enabled DMZ: 10.112.42.15\n"
        "MAC: 5A:78:3E:7E:00\n"
        "ISP: Rostelekom\n"
        "DNS: 8.8.8.8\n"
        "ALT DNS: 1.1.1.8.1\n"
        "Dlink WAN: 100.23.10.15\n"
        "GATEWAY: 192.168.0.1\n"
        "SUBNET MASK: 255.255.0.255\n"
        "UDP OPEN PORTS: 8080,80\n"
        "TCP OPEN PORTS: 443\n"
        "ROUTER VENDOR: ERICSSON\n"
        "DEVICE VENDOR: WIN32-X\n"
        "CONNECTION TYPE: Ethernet\n",
    ) # ЭТО ШУТКА, ЕСЛИ ЧТО!

# Реакция на текст
@bot.message_handler(content_types=["text"])
def echo_all(message):
    responses = [

```

```

    "Мой синтаксический анализатор сегодня сбоит, я не понимаю тебя.",
    "А?",
    "Межауши межазаложило. Ты точно сказал что-то, что я должен понять?",
]
bot.reply_to(message, random.choice(responses))

bot.infinity_polling()

```

Экранные формы с примерами выполнения программы

